6. Iterate through the table and column data, leveraging our helper functions to format the SQL database input.

```
metadata = {}
table_list = []

for row in table_schemas.select("table_name").distinct().collect():
  tbl = row['table_name']
  table_list.append(tbl)
  meta = table_def(tbl) + '\n\n' + table_records(tbl)
  metadata[tbl] = meta
```

Figure 8.25 – Iterating through the tables and leveraging the helper functions

7. We now have all of our data ready to be able to create a SQL database for OpenAI to talk to. You will need to edit `endpoint_http_path` to match the path of an active SQL warehouse in your workspace. The database is passed to both the default OpenAI model and the GPT 3.5 model.

```
from sqlalchemy.engine import create_engine
from langchain.sql_database import SQLDatabase
from langchain_experimental.sql import SQLDatabaseChain

table = table_list

databricks_token = dbutils.notebook.entry_point.getDbutils().notebook().getContext().apiToken().get()
workspace_url = spark.conf.get("spark.databricks.workspaceUrl")
# Obtain this from a SQL endpoint under "Connection Details", HTTP Path
endpoint_http_path = "/sql/1.0/warehouses/5ab5dda58c1ea16b"

engine = create_engine(
  f"databricks+connector://token:{databricks_token}@{workspace_url}:443/{database_name}",
  connect_args={"http_path": endpoint_http_path, "catalog": catalog}
)

db = SQLDatabase(engine, schema=None, include_tables=table)
db_chain = SQLDatabaseChain.from_llm(llm, db, verbose=True)
chat_chain = SQLDatabaseChain.from_llm(chat, db, verbose=True)
```

Figure 8.26 – Create a database for OpenAI to query containing only the information we provided

With the setup complete, we can now interact with our SQL chatbot models! Let's start with a basic question: *Which store sold the most?*